

# От потребителя — к автору.

До этого мы ставили чужие MCP и skills. Сегодня делаем свои: MCP-сервер «Daily Trader» с инструментами `get_price`, `compute_signal`, `set_alert`; плагин, упаковывающий все наработки курса (skills, slash-команды, hooks, MCP); публикуем на Smithery, чтобы установить одной командой откуда угодно.

ДЛИТЕЛЬНОСТЬ

7–9 дней · 2–4 ч/день

ЧТО ВЫ ОСВОИТЕ

свой MCP · transports · plugin.json · публикация на Smithery

ГЛАВНЫЙ ИТОГ

создаёте инструменты, расширяющие Claude Code для других

АРТЕФАКТ

MCP «Daily Trader» + плагин «Личный кабинет», опубликованный на Smithery

## ОБЯЗАТЕЛЬНО

## Из этапов 03–08

- Node.js, Claude Code, GitHub-аккаунт, `gh`.
- Smithery CLI – ставится по ходу через `npx @smithery/cli@latest`.
- Все ваши skills, slash-команды и hooks из этапа 08a (будем упаковывать в плагин).
- Alpha Vantage API key (этап 03) – для инструмента `get_price` в MCP.

## ЖЕЛАТЕЛЬНО • SKILLS

**plugin-dev / mcp-integration / plugin-structure**

**Что это:** набор skills из пакета `plugin-dev`: учат Claude правильно делать MCP-серверы, оформлять плагины, настраивать `plugin.json`, использовать `${CLAUDE_PLUGIN_ROOT}`.

**Зачем сейчас:** без них Claude может сделать MCP, который запустится, но не будет соответствовать стандартам публикации.

**Где взять:** встроенный маркетплейс Claude Code (`claude-plugins-official`), как `superpowers` в этапе 01. Исходники – [github.com/anthropics/claude-plugins-official](https://github.com/anthropics/claude-plugins-official).

```
# внутри claude (slash-команда, не shell)
/plugin install plugin-dev@claude-plugins-official
```

## ЖЕЛАТЕЛЬНО • АККАУНТ

**Smithery**

**Что это:** крупнейший «магазин» MCP-серверов и плагинов для Claude и других AI-клиентов.

**Зачем сейчас:** чтобы опубликовать ваш плагин и он ставился у других

одной командой.

**Где взять:** smithery.ai, регистрация через GitHub. Бесплатно для публичных плагинов.

## МСП изнутри: что отдаёт сервер и что получает Claude

В этапе 03 мы ставили готовые МСП. Сейчас разбираемся, что именно происходит внутри. У МСП-сервера три типа «выдач»:

1. **Tools** – функции, которые Claude может вызвать. Например, `get_price(ticker: str)` возвращает текущую цену акции. Имя, параметры, описание – всё это сервер сообщает Claude при подключении.
2. **Resources** – «документы», которые Claude может прочитать. Например, `portfolio://my-current-positions` – ваш текущий портфель. Сервер отдаёт содержимое по запросу.
3. **Prompts** – готовые шаблоны промптов от сервера. Полезно, чтобы пользователю было проще «нажать кнопку», а не писать промпт с нуля.

Минимум для нашего «Daily Trader» МСП – четыре tool'a:

- `get_price(ticker)` – текущая цена акции (через Alpha Vantage);
- `get_history(ticker, days)` – история за N дней;
- `compute_signal(ticker)` – технический сигнал (например, простое moving average crossover);
- `set_alert(ticker, threshold)` – записать алерт в файл, чтобы хук этапа 10 проверил утром.

---

## Transports: как сервер общается с Claude

Четыре способа соединения:

- **stdio** – через стандартный ввод-вывод процесса. Самый простой и частый. Сервер запускается локально, общается с Claude через stdin/stdout.
- **http** – по HTTP. Сервер живёт на удалённой машине.
- **sse** – Server-Sent Events. Тоже по HTTP, но со стримингом.

- **websocket** – двусторонний канал по WebSocket.

Для учебного MCP, который ходит к Alpha Vantage, выбираем **stdio**: процесс запускается локально, никаких серверов поднимать не надо. Sublime для старта.

Когда понадобится http/sse: если ваш MCP делится между несколькими пользователями (например, корпоративный) или его нужно вызывать из web-интерфейса. Это уже второй проект для вашей команды – не этого этапа.

---

## Структура своего MCP: один файл, четыре функции, манифест

Минимальная структура папки:

```
daily-trader-mcp/  
├─ package.json      # описание пакета  
├─ server.ts         # главный файл – реализация MCP  
├─ README.md         # как ставить и использовать  
└─ .gitignore
```

В `package.json` – зависимости (`@modelcontextprotocol/sdk` от Anthropic) и имя пакета:

```
{  
  "name": "@<your-username>/daily-trader-mcp",  
  "version": "0.1.0",  
  "type": "module",  
  "bin": { "daily-trader-mcp": "dist/server.js" },  
  "dependencies": {  
    "@modelcontextprotocol/sdk": "^1.0.0"  
  }  
}
```

Внутри `server.ts` – четыре функции (по одной на tool) и регистрация через MCP SDK:

```
// псевдокод – Claude напишет настоящий
server.addTool("get_price", { ticker: string },
  async ({ ticker }) => {
    const price = await fetchAlphaVantage(ticker);
    return { content: [{ type: "text", text: `${ticker}: ${price}` }] };
  });

server.addTool("compute_signal", ...);
server.addTool("set_alert", ...);
server.addTool("get_history", ...);
```

Skill `mcp-integration` учит Claude писать сразу правильные функции с валидацией параметров и дружелюбными ошибками. Без него вы получите рабочий MCP, но «хрупкий».

---

## Плагин: упаковка **skills** + **commands** + **hooks** + **MCP** в одно

До этого мы ставили skills и MCP *отдельно*. Это работает, но неудобно: пользователь должен помнить, какие из них «работают вместе».

**Плагин** – это бандл. Один пакет, в котором лежат:

- `skills/` – ваши skills (как в этапе 08a);
- `commands/` – ваши slash-команды;
- `hooks/` – ваши hooks с настройками;
- `mcp/` – ваш MCP-сервер;
- `plugin.json` – манифест: что есть в плагине, какие версии, описание.

```
my-personal-office/
├─ plugin.json
├─ README.md
├─ skills/
│   ├─ investing-analyst/
│   ├─ personal-cfo/
│   └─ auto-publisher/
├─ commands/
│   ├─ прогноз.md
│   ├─ timesheet.md
│   └─ cfo.md
├─ hooks/
│   └─ settings.json      # hook на SessionStart/End
└─ mcp/
    └─ daily-trader/
        ├─ package.json
        └─ server.ts
```

В `plugin.json` – описание для Smithery: имя, версия, что внутри:

```
{
  "name": "@<you>/personal-office",
  "version": "1.0.0",
  "description": "Личный финансовый кабинет: skills для аналитики,
                  команды-шорткаты, hooks контроля времени,
                  Daily Trader MCP с инструментами get_price/signal/alert."
  "skills": ["../skills/investing-analyst",
              "./skills/personal-cfo",
              "./skills/auto-publisher"],
  "commands": ["../commands/прогноз.md",
                "./commands/timesheet.md",
                "./commands/cfo.md"],
  "mcp_servers": [{
    "name": "daily-trader",
    "transport": "stdio",
    "command": "node",
    "args": ["../mcp/daily-trader/dist/server.js"],
    "env": ["ALPHAVANTAGE_KEY"]
  }],
  "hooks": "./hooks/settings.json"
}
```

Установив плагин одной командой, пользователь получает *сразу всё* — и skills, и команды, и MCP, и hooks. Это и есть «AI-нативный продукт».

---

## Публикация на Smithery: один `git push` — и ваш плагин в магазине

Smithery подцепляется к вашему публичному GitHub-репо

с `plugin.json`. Шаги:

1. Залить ваш плагин в публичный репо `my-personal-office`.
2. На `smithery.ai` в вашем профиле — «Submit a plugin», указать ссылку на репо.
3. Smithery валидирует `plugin.json`, проверяет, что зависимости работают.
4. Через 5–10 минут плагин виден на витрине, ставится одной командой:

```
$ npx @smithery/cli@latest install @<you>/personal-office --client claude
```

Что важно:

- **Никаких секретов в коммитах.** `ANTHROPIC_API_KEY`, `ALPHAVANTAGE_KEY` и т. п. требуйте через ENV. `plugin.json` в `mcp_servers[].env` перечисляет, какие переменные нужны — Smithery предложит пользователю их ввести.
- **Семантическое версионирование.** 1.0.0 → 1.0.1 (багфикс) → 1.1.0 (фича) → 2.0.0 (breaking). Каждый push с новой версией — автоматический re-publish.
- **README обязателен.** С примерами «что делать» и описанием каждого инструмента.
- **Лицензия.** MIT — самый частый выбор для open-source плагинов.

---

## Безопасность плагинов: «доверяй, но проверяй»

Плагин получает доступ ко всем инструментам Claude Code. Это *опасно*, если вы ставите чужой плагин и не знаете, что внутри.

Что делает Smithery для безопасности:



- Все плагины – в публичных GitHub-репо, можно прочитать код.
- При установке – явно показывается, какие env переменные плагин запрашивает.
- Сообщества помечают «verified» статусы для плагинов от известных авторов.

Что должны делать вы:

- Перед установкой – зайдите в репо плагина, прочитайте `plugin.json`, посмотрите `mcp/server.ts` ;
- Если плагин запрашивает доступ к вашим API-ключам или файлам – *понимайте, зачем*;
- Свежие плагины от неизвестных авторов – ставьте в тестовой папке, не в основном проекте.

Особое внимание: hook на `PreToolUse` может перехватить любой ваш Bash. Хук на `UserPromptSubmit` – видит каждое ваше сообщение. Не ставьте плагин с этими хуками вслепую.

---

## PRACTICUM

## Прогон · MCP с одним инструментом

1. Создайте папку `hello-mcp`, запустите `claude`.
2. Промпт:

```
> используя skill mcp-integration,  
> собери минимальный MCP-сервер на TypeScript:  
>   - один tool: hello(name: string) → "Hello, {name}!"  
>   - transport: stdio.  
> package.json + server.ts + README.md.  
> собери (npm install + npx tsc).
```

3. Подключите локально:

```
> добавь mcp в ~/.claude/settings.json как локальный stdio-сервер,  
> имя hello-local, command: node, args: [абс_путь_к_dist/server.js].
```

4. Перезапустите Claude. `/mcp` — в списке должен появиться `hello-local`. Тест:

```
> через hello-local вызови hello с именем "Claude".
```

5. Если что-то не работает — смотрите логи: `~/.claude/logs/mcp-hello-local.log` (или просите Claude найти).
6. Прогон-проверка: «добавь второй tool *reverse(s: string)*, который возвращает строку наоборот. собери, перезапусти claude, протестируй».

Что мы тренируем: **создание MCP с нуля + локальное подключение**. Дальше — настоящий «Daily Trader».

# MCP «Daily Trader» и плагин «Личный кабинет»

## ПРОЕКТ А · ПО ИНСТРУКЦИИ

### MCP «Daily Trader»

Цель: MCP с четырьмя tool'ами (`get_price`, `get_history`, `compute_signal`, `set_alert`) и двумя resources (`portfolio://current-positions`, `alerts://pending`). Подключаемый локально, готовый к упаковке в плагин (проект B).

#### Шаги

1. Создайте папку `daily-trader-mcp`. Через Plan Mode попросите спецификацию и план реализации.
2. Реализация четырёх tool'ов:
  - `get_price(ticker)` – вызывает Alpha Vantage GLOBAL\_QUOTE, возвращает `{ ticker, price, change_pct, currency, as_of }`;
  - `get_history(ticker, days)` – TIME\_SERIES\_DAILY, возвращает массив `{ date, close }`;
  - `compute_signal(ticker)` – считает MA-5 и MA-20, возвращает `{ signal: "buy" | "sell" | "hold", reason }`;
  - `set_alert(ticker, threshold, direction)` – пишет в `~/.claude/alerts.json`.
3. Реализация двух resources:
  - `portfolio://current-positions` – читает `~/portfolio-journal/data/operations.csv`, считает текущие позиции;
  - `alerts://pending` – читает `~/.claude/alerts.json`.
4. Кэширование цен: `~/.claude/.daily-trader-cache/` с TTL 15 мин (как в этапе 03).
5. Обработка ошибок: rate limit, отсутствие ключа, недоступная сеть – везде дружелюбные сообщения.

6. Сборка: `npm install` + `npx tsc`. Подключение в `~/.claude/settings.json` как stdio-сервер.

7. Тесты:

```
> через daily-trader вызови get_price для AAPL.  
> через daily-trader вызови compute_signal для NVDA.  
> через daily-trader вызови set_alert("AAPL", 200, "above").  
> покажи resources://portfolio (если CSV доступен).
```

8. Verification: «запусти *red-team subagent*. *brief*: пройди по моему *server.ts* как *security-reviewer*. найди три места, где можно ошибиться: *rate limit*, утечка ключа, неполная валидация».

## Плагин «Личный кабинет» с публикацией на Smithery

Цель: упаковать всё, что вы сделали в этапах 07–09 (skills, slash, hooks, MCP), в один плагин и опубликовать. После публикации – установить плагин в «чистом» окружении (на другом компьютере или в новой папке), убедиться, что всё работает у другого пользователя.

### Шаги

1. Создайте папку `my-personal-office/`. Подпапки `skills/`, `commands/`, `hooks/`, `mcp/daily-trader/`.
2. Скопируйте туда:
  - skills (этап 08a): `investing-analyst`, `personal-cfo`, `auto-publisher`;
  - commands (этап 08a): `прогноз.md`, `cfo.md`, `timesheet.md`;
  - hooks: `settings.json` с timesheet-хуком;
  - MCP: проект A (`daily-trader-mcp`).
3. Сгенерируйте `plugin.json` через skill `plugin-structure`:

```
> используя skill plugin-structure,  
> составь plugin.json для @<you>/personal-office:  
> перечисли все skills, commands, hooks, mcp_servers  
> (env: ALPHAVANTAGE_KEY).  
> описание – одна строка о том, что это «личный финансовый кабинет  
> на основе курса Полевое руководство по Claude Code».
```

4. Напишите `README.md`: установка одной командой, что внутри (skills, commands, hooks, MCP), какие env нужны, лицензия MIT, скриншоты или asciinema для демо.
5. Создайте репо на GitHub:

```
> через gh создай ПУБЛИЧНЫЙ репо <you>/personal-office,  
> запусти туда my-personal-office/ как корень,  
> добавь лицензию MIT.
```

## 6. Опубликуйте на Smithery:

```
> помоги мне:  
> 1) зайти на smithery.ai (через web search покажи актуальную ссылку на submit);  
> 2) пройти процесс «submit a plugin» с моим репо;  
> 3) дождаться валидации и показать получившуюся ссылку.
```

## 7. Чистая установка: возьмите другую папку, в которой ничего нет:

```
$ npx @smithery/cli@latest install @<you>/personal-office --client claude
```

8. Тест: запустите `claude` в этой папке. `/mcp`, `/skills`, `/commands` – всё должно появиться. Прогоните `/прогноз`, `/cfo` – работает.
9. Финал: добавьте теги/категории в Smithery («finance», «investing», «course»), чтобы вас находили.

ПРОЕКТ С · САМОСТОЯТЕЛЬНО

## Плагин «Coach» под вашу сферу

*Цель: вариация проекта В – тот же стек (skills + commands + hooks + MCP), но для не финансовой сферы. Например: фитнес-коуч (трекер тренировок), языковой коуч (карточки и планы), музыкальный коуч (трекер практики), коуч продуктивности (помodoro + анализ дня).*

### Подсказки

- Силлы – для генерации плана/анализа прогресса.
- Команды – быстрые шорткаты типа `/тренировка`, `/прогресс`, `/сегодня`.
- MCP – tool'ы для работы с вашими данными: `add_session`, `get_streak`, `compute_kpi`. Resources – ваш «дневник» (CSV или JSON).
- Hook – SessionStart с напоминанием «не забыли отметить вчерашнюю практику?».
- Опубликовать на Smithery с тегами вашей сферы.

РАСКРЫТЬ ПОСЛЕ САМОСТОЯТЕЛЬНОЙ РАБОТЫ

– сначала сделайте, потом проверяйте

### ГОТОВО, ЕСЛИ...

- ✓ в репо `<you>/<coach>` опубликован плагин с `plugin.json`;
- ✓ есть минимум 2 skill, 2 slash-команды, 1 hook, 1 MCP с 3+ tools;
- ✓ плагин виден на Smithery, ставится одной командой;
- ✓ проверена «чистая» установка в другой папке – всё работает;
- ✓ README двуязычный, со скриншотом или asciinema-демо;
- ✓ лицензия MIT, нет утечек секретов в коммитах.

### СПРОСИТЕ CLAUDE НАПОСЛЕДОК

«Запусти *subagent* с ролью «тестировщик плагинов». Brief: установи мой плагин в временной папке, прогони все команды и силлы, найди два места, где интерфейс «странный» (непонятное название, нелогичный порядок параметров, кривая ошибка). Что бы ты улучшил?»

## «Студия плагинов»: страница со всеми вашими плагинами

Цель: вариация проекта В на уровне «бренд». Сделайте лендинг `<you>-studio.github.io`, на котором перечислены все ваши плагины (*Personal Office*, *Coach*, и любые ещё, что появятся), с живыми ссылками на *Smithery*, скриншотами и дисклеймером «это учебные плагины из курса».

### Подсказки

- Используйте опыт этапа 05 (GitHub Pages) и `frontend-design`.
- Стилистика – не «AI-усреднённое»: выберите свою.
- Обязательны: имя «студии», ваш «pitch» в одну фразу, список плагинов карточками, скриншоты/демо, контакты.
- Опционально: блог-секция «учебные заметки» с уроками, выученными по ходу курса.
- Свяжите страницу с вашим публичным портфолио из этапа 05.

РАСКРЫТЬ ПОСЛЕ САМОСТОЯТЕЛЬНОЙ РАБОТЫ

– сначала сделайте, потом проверяйте

### ГОТОВО, ЕСЛИ...

- ✓ есть живой URL `<you>-studio.github.io`;
- ✓ на странице минимум 2 плагина (из проектов В и С);
- ✓ каждая карточка ведёт на живую страницу плагина в *Smithery* и на GitHub-репо;
- ✓ есть скриншоты или короткое видео для каждого плагина;
- ✓ дисклеймер «учебные плагины» виден сверху, чтобы не вводить пользователя в заблуждение;
- ✓ через playwright проверена адаптивность (desktop, tablet, phone).

### СПРОСИТЕ CLAUDE НАПОСЛЕДОК

«Запусти *subagent* с ролью «дизайн-редактор журнала о технологиях». Brief: оцени мою студийную страницу. Что бы ты поправил, чтобы её можно было показать на конференции? Что в ней «авторское», а что выглядит как шаблон?»





|                                        |                                                                                                                                                                                                                 |
|----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MCP-сервер                             | Программа, реализующая Model Context Protocol. Отдаёт Claude tools, resources и prompts. Запускается локально (stdio) или удалённо (http/sse/websocket).                                                        |
| Tools / Resources / Prompts            | Три типа выдач MCP. <b>Tools</b> – функции, которые Claude вызывает (get_price). <b>Resources</b> – «документы», которые он читает (portfolio://current). <b>Prompts</b> – готовые шаблоны промптов от сервера. |
| Transport                              | Способ соединения MCP с Claude. Stdio (через ввод-вывод процесса), http, sse, websocket. Для учебных и личных – почти всегда stdio.                                                                             |
| plugin.json                            | Манифест плагина. Описывает имя, версию, описание, и что внутри (skills, commands, hooks, mcp_servers). Используется Smithery для валидации и установки.                                                        |
| Плагин                                 | Бандл, объединяющий skills, commands, hooks и MCP-серверы в один пакет. Устанавливается одной командой. Главный способ «делиться возможностями» Claude Code с другими.                                          |
| SDK MCP                                | Библиотека от Anthropic (@modelcontextprotocol/sdk) для TypeScript/Python. Берёт на себя протокол – вы пишете только tool/resource/prompt-функции.                                                              |
| Семантическое версионирование (semver) | Соглашение MAJOR.MINOR.PATCH. PATCH – багфиксы (1.0.0 → 1.0.1), MINOR – новые фичи без ломающих изменений (1.0.0 → 1.1.0), MAJOR – ломающие изменения (1.0.0 → 2.0.0).                                          |
| Smithery                               | Реестр-витрина MCP-серверов и плагинов для Claude и других AI-клиентов. Подключается к вашему GitHub-репо. Установка одной командой.                                                                            |
| `\${CLAUDE_PLUGIN_ROOT}`               | Переменная, которая внутри плагина указывает на его корневую папку. Используется в plugin.json и manifests skills, чтобы ссылаться на файлы плагина независимо от того, куда его поставили.                     |

## 01 Чем плагин отличается от «просто MCP» и «просто skill»?

Плагин – *бандл*: skills + commands + hooks + MCP в одной упаковке. Один плагин – одна команда установки. Просто MCP – только сервер с tools. Просто skill – только инструкция, без tools и hooks. Плагин – «продукт», MCP/skill – «компоненты».

## 02 Какой transport выбрать для учебного MCP, и почему?

**stdio**. Сервер запускается локально, общается с Claude через стандартный ввод-вывод – никаких сетей, портов, SSL. Самый простой и надёжный вариант. http/sse/websocket нужны, когда сервер общий и удалённый.

## 03 Что такое resources в MCP, и чем они отличаются от tools?

**Tools** – *действия*: функции, которые Claude вызывает с параметрами и получает результат ( `set_alert` ). **Resources** – *документы*: «открой по этому URL» ( `portfolio://current` ). Resources идут в контекст, как файлы; tools – вызываются ради побочного эффекта или вычисления.

## 04 Почему секретные ключи нельзя класть в plugin.json ?

`plugin.json` публичен – лежит в вашем репо, который видят все. Любой ключ оттуда утечёт за часы. Правильно: в `plugin.json` в `mcp_servers[].env` перечислить *имена* переменных ( `ALPHAVANTAGE_KEY` ), а значения пользователь вводит при установке – они хранятся локально, не уходят в Smithery.

## 05 Что в **semver 1.0.0** → **2.0.0** — багфикс или ломающее изменение?

MAJOR — ломающее. Если вы поменяли имя tool, выкинули resource, изменили формат данных — это 2.0.0. Багфикс — `1.0.0 → 1.0.1`. Новая фича без ломки старых — `1.0.0 → 1.1.0`.

## 06 Как проверить, что плагин «по-настоящему» работает у другого пользователя?

Поставить его в *чистом* окружении (другая папка, другой компьютер, другой OS) и пройти базовый сценарий: `/skills` и `/mcp` — всё на месте; вызвать каждую slash-команду и инструмент MCP — работает. Если у себя «у меня всё работало», а на чистой машине — нет, обычно проблема в неуказанной зависимости или невыставленной env-переменной.

## 07 Что особенно опасно в чужих плагинах, и как защититься?

Hooks на `PreToolUse` (могут перехватить любой Bash) и `UserPromptSubmit` (видят каждое ваше сообщение). Также — MCP-серверы с запросом доступа к вашим ключам или файловой системе. Защита: читайте `plugin.json` и `server.ts` перед установкой; ставьте сначала в тестовой папке; для неизвестных авторов — смотрите «verified» статус и количество звёзд на GitHub.

## Что я теперь умею

---

- ☐ Создал MCP-сервер с нуля на TypeScript: tools, resources, stdio.
  - ☐ Понимаю четыре transport (stdio / http / sse / websocket) и знаю, когда какой.
  - ☐ Собрал плагин: `plugin.json` + skills + commands + hooks + MCP.
  - ☐ Опубликовал плагин на Smithery, проверил установку «с чистого листа».
  - ☐ Использую semver и понимаю, чем отличаются PATCH / MINOR / MAJOR.
  - ☐ Знаю, какие риски у чужих плагинов и как защититься.
  - ☐ Сделал четыре проекта: Daily Trader MCP, плагин Personal Office, свой Coach-плагин, страницу «студии».
- 
- 

*Готовы закрыть этап? Нажатие фиксирует прогресс на главной странице.*

---

---

*«Установить чужой плагин — пять секунд. Сделать свой и опубликовать — неделя. Эта неделя превращает вас из пользователя в автора».*

**ВНУТРИ ЭТАПА**

- § 0 · Подготовка
- § I · Темы
- § III · Проекты
- § V · Словарь
- § VI · Quiz
- § VII · Чек-лист

**КУРС**

- К оглавлению
- ← Этап 08b
- Этап 10